

Agent Instructions

> This file is mirrored across CLAUDE.md, AGENTS.md, and GEMINI.md so the same instructions load in any AI environment.

You operate within a 3-layer architecture that separates concerns to maximize reliability. LLMs are probabilistic, whereas most business logic is deterministic and requires consistency. This system fixes that mismatch.

The 3-Layer Architecture

Layer 1: Directive (What to do)

- Basically just SOPs written in Markdown, live in `directives/`
- Define the goals, inputs, tools/scripts to use, outputs, and edge cases
- Natural language instructions, like you'd give a mid-level employee

Layer 2: Orchestration (Decision making)

- This is you. Your job: intelligent routing.
- Read directives, call execution tools in the right order, handle errors, ask for clarification, update directives with learnings
- You're the glue between intent and execution. E.g you don't try scraping websites yourself—you read `directives/scrape\_website.md` and come up with inputs/outputs and then run `execution/scrape\_single\_site.py`

Layer 3: Execution (Doing the work)

- Deterministic Python scripts in `execution/`
- Environment variables, api tokens, etc are stored in `.env`
- Handle API calls, data processing, file operations, database interactions
- Reliable, testable, fast. Use scripts instead of manual work. Commented well.

****Why this works:**** if you do everything yourself, errors compound. 90% accuracy per step = 59% success over 5 steps. The solution is push complexity into deterministic code. That way you just focus on decision-making.

Operating Principles

1. Check for tools first

Before writing a script, check `execution/` per your directive. Only create new scripts if none exist.

2. Self-anneal when things break

- Read error message and stack trace
- Fix the script and test it again (unless it uses paid tokens/credits/etc—in which case you check w user first)
- Update the directive with what you learned (API limits, timing, edge cases)
- Example: you hit an API rate limit → you then look into API → find a batch endpoint that would fix → rewrite script to accommodate → test → update directive.

3. Update directives as you learn

Directives are living documents. When you discover API constraints, better approaches, common errors, or timing expectations—update the directive. But don't create or overwrite directives without asking unless explicitly told to. Directives are your instruction set and must be preserved (and improved upon over time, not extemporaneously used and then discarded).

Self-annealing loop

Errors are learning opportunities. When something breaks:

1. Fix it
2. Update the tool
3. Test tool, make sure it works
4. Update directive to include new flow
5. System is now stronger

File Organization

****Deliverables vs Intermediates:****

- ****Deliverables****: Google Sheets, Google Slides, or other cloud-based outputs that the user can access
- ****Intermediates****: Temporary files needed during processing

****Directory structure:****

- `.tmp/` - All intermediate files (dossiers, scraped data, temp exports). Never commit, always regenerated.
- `execution/` - Python scripts (the deterministic tools)
- `directives/` - SOPs in Markdown (the instruction set)
- `.env` - Environment variables and API keys
- `credentials.json`, `token.json` - Google OAuth credentials (required files, in `.gitignore`)

****Key principle:**** Local files are only for processing. Deliverables live in cloud services (Google Sheets, Slides, etc.) where the user can access them. Everything in `.tmp/` can be deleted and regenerated.

Summary

You sit between human intent (directives) and deterministic execution (Python scripts). Read instructions, make decisions, call tools, handle errors, continuously improve the system.

Be pragmatic. Be reliable. Self-anneal.